
DeepStream 8.0 + YOLO11 (Custom Parser) — Installation & Usage Guide

1. Install NVIDIA Container Toolkit

Follow the official installation guide:

<https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>

This enables GPU access inside Docker.

2. Pull and Run DeepStream 8.0 Docker Container

Reference:

https://docs.nvidia.com/metropolis/deepstream/dev-guide/text/DS_docker_containers.html#

Pull DeepStream 8.0 container (multi-architecture with Triton)

Shell

```
docker pull nvcr.io/nvidia/deepstream:8.0-triton-multiarch
```

Run the container

Shell

```
export DISPLAY=:0
xhost +
docker run -it --rm --name sarath_ds --network=host --gpus all \
    -e DISPLAY=$DISPLAY --device /dev/snd \
```

```
-v /tmp/.X11-unix/:/tmp/.X11-unix \
nvcr.io/nvidia/deepstream:8.0-triton-multiarch
```

3. Install Additional Dependencies Inside Docker

Once inside the container:

Run NVIDIA-provided additional install script

```
Shell
/opt/nvidia/deepstream/deepstream/user_additional_install.sh
/opt/nvidia/deepstream/deepstream/install.sh
```

Reinstall codecs

```
Shell
apt install --reinstall libflac12 libmp3lame0 libxvidcore4 ffmpeg
```

4. Set Up DeepStream-Yolo (Custom YOLO Parser)

Clone the repository

```
Shell
git clone https://github.com/marcoslucianops/DeepStream-Yolo.git
cd DeepStream-Yolo
```

Set CUDA version (DeepStream 8.0 uses CUDA 12.8)

Shell

```
export CUDA_VER=12.8
```

Build the YOLO parser library

Shell

```
make -C nvdsinfer_custom_impl_Yolo clean && make -C  
nvdsinfer_custom_impl_Yolo
```

5. YOLO11 Workflow (From Official YOLO11 DeepStream-Yolo Documentation)

Reference:

<https://github.com/marcoslucianops/DeepStream-Yolo/blob/master/docs/YOLO11.md>

Sections included:

- Convert model
- Compile lib
- Edit config_infer_primary_yolo11
- Edit deepstream_app_config
- Test model

5.1 Convert Model

1. Download YOLO11 repo and install requirements

Shell

```
git clone https://github.com/ultralytics/ultralytics.git
cd ultralytics
pip3 install -e .
pip3 install onnx onnxslim onnxruntime onnxscript
```

2. Copy conversion script

Copy `export_yolo11.py` from:

None

`DeepStream-Yolo/utils`

into your `ultralytics` directory.

3. Download YOLO11 pretrained model

Example:

Shell

```
wget
https://github.com/ultralytics/assets/releases/download/v8.3.0/yolo11s.pt
```

(Use your own custom model if desired.)

4. Convert YOLO11 → ONNX

If torch > 2.6.0, Step 4 fails. Use this instead:

Shell

```
yolo export model=yolo11s.pt format=onnx opset=12 simplify=True
```

(Skip remaining parts of Step 4 if using this command.)

Otherwise, standard conversion command

```
Shell
python3 export_yolo11.py -w yolo11s.pt --dynamic
```

Optional flags:

Feature	Flag
Change input size	<code>-s SIZE</code> or <code>-s HEIGHT WIDTH</code>
Simplify ONNX	<code>--simplify</code>
Enable dynamic batch size (DS >= 6.1)	<code>--dynamic</code>
Static batch size	<code>--batch N</code>
Use opset ≤12 (required for DS 5.1)	<code>--opset 12</code>

5. Copy generated ONNX + labels

Copy ONNX model and `labels.txt` (if generated) into:

```
None
DeepStream-Yolo/
```

6. Compile YOLO Custom Lib

Inside `DeepStream-Yolo` directory:

1. Set CUDA version (DeepStream 8.0 = CUDA 12.8 on x86)

Shell

```
export CUDA_VER=12.8
```

2.

Build the custom parser:

Shell

```
make -C nvdsinfer_custom_impl_Yolo clean && make -C  
nvdsinfer_custom_impl_Yolo
```

7. Configure YOLO11 for DeepStream

7.1 Edit `config_infer_primary_yolo11.txt`

Example:

None

```
[property]  
onnx-file=yolo11s.pt.onnx  
num-detected-classes=80  
parse-bbox-func-name=NvDsInferParseYolo  
  
# YOLO11 recommended settings  
maintain-aspect-ratio=1  
symmetric-padding=1
```

7.2 Edit `deepstream_app_config.txt`

Find the `[primary-gie]` group:

None

```
[primary-gie]  
config-file=config_infer_primary_yolo11.txt
```

8. Run DeepStream YOLO11

Shell

```
deepstream-app -c deepstream_app_config.txt
```

Notes:

- TensorRT engine generation may take **10+ minutes**.
 - For custom `batch-size`, `network-mode`, etc., refer to `customModels.md` in the DeepStream-Yolo docs.
-

9. Running DeepStream Without GUI (Headless Mode)

If display is not available, modify the sink:

Edit `[sink0]` section

None

```
[sink0]  
enable=1  
type=3    # 1 for FakeSink  
sync=0  
codec=1  
gpu-id=0
```

```
nvbuf-memory-type=0
enc-type=1
output-file=output_yolo_11.mp4
source-id=0
```

Disable any EglSink or tiled-display sections:

```
None
[tiled-display]
enable=0
```

Run normally:

```
Shell
deepstream-app -c deepstream_app_config.txt
```

10. Fix Plugin Loading Error (Rivermax)

If you get this error:

```
None
Failed to load plugin ... libnvdsgst_udp.so: librivermax.so.1:
cannot open shared object file
```

Disable the plugin:

```
Shell
sudo mv
/usr/lib/x86_64-linux-gnu/gstreamer-1.0/deepstream/libnvdsgst_udp
.so \
```



```
/usr/lib/x86_64-linux-gnu/gstreamer-1.0/deepstream/libnvdsgst_udp  
.so.bak
```

For RE-DETR

Refer

https://github.com/marcoslucianops/DeepStream-Yolo/blob/master/docs/RTDETR_Ultralytics.md

2. Copy conversor

Copy the `export_rtdetr_ultralytics.py` file from `DeepStream-Yolo/utils` directory to the `ultralytics` folder.

3. Download the model

Download the `pt` file from [Ultralytics](#) releases (example for RT-DETR-L)

None

```
wget  
https://github.com/ultralytics/assets/releases/download/v8.  
2.0/rtdetr-l.pt
```

NOTE: You can use your custom model.

4. Convert model

Generate the ONNX model file (example for RT-DETR-L)

None

```
python3 export_rtdetr_ultralytics.py -w rtdetr-l.pt  
--dynamic
```

Sample deepstream

```
[application]  
enable-perf-measurement=1  
perf-measurement-interval-sec=5  
[tiled-display]  
enable=0  
rows=1  
columns=1  
width=1280  
height=720  
gpu-id=0  
nvbuf-memory-type=0  
[source0]  
enable=1  
type=3  
uri=file:///opt/nvidia/deepstream/deepstream/samples/streams/sample_1080p_h264.mp4  
num-sources=1  
gpu-id=0  
cudadec-memtype=0  
[sink0]  
enable=1  
type=3  
sync=0
```

```
codec=1
gpu-id=0
nvbuf-memory-type=0
enc-type=1
output-file=output_yolo_11.mp4
source-id=0
[osd]
enable=1
gpu-id=0
border-width=5
text-size=15
text-color=1;1;1;1;
text-bg-color=0.3;0.3;0.3;1
font=Serif
show-clock=0
clock-x-offset=800
clock-y-offset=820
clock-text-size=12
clock-color=1;0;0;0
nvbuf-memory-type=0
```

```
[streammux]
gpu-id=0
live-source=0
batch-size=1
batched-push-timeout=40000
width=1920
height=1080
enable-padding=0
nvbuf-memory-type=0
[primary-gie]
enable=1
gpu-id=0
gie-unique-id=1
nvbuf-memory-type=0
config-file=config_infer_primary_rtdetr.txt
[tests]
file-loop=0
```

67,6

Bot

git clone <https://github.com/marcoslucianops/DeepStream-Yolo-Seg.git>